

Smart Real-Time Monitoring & Inspection Platform

Implementation Plan v2 — SIH Problem Statement #26095 (MoSJE / DoSJE)

Scope for this phase: Web application only, developed and run on local machine (localhost). No native mobile app for now — the “field/inspection” role also uses the web app via browser (mobile-responsive), so nothing is lost functionally.

PART 1 — Project Overview

A centralized **Django-powered web platform** giving DoSJE real-time visibility into projects/institutes/NGOs running under its schemes — live CCTV feeds, random video-conference check-ins, AI-driven random inspection assignment, geo-tagged inspection reports, anomaly/attendance analytics, and citizen-facing transparency tools.

Everything below is designed to run entirely on **localhost** during development (Docker Compose spins up Postgres, Redis, Django, the AI microservice, and MediaMTX all on your machine — no cloud dependency needed until you're ready to deploy).

PART 2 — Tech Stack (Web-only)

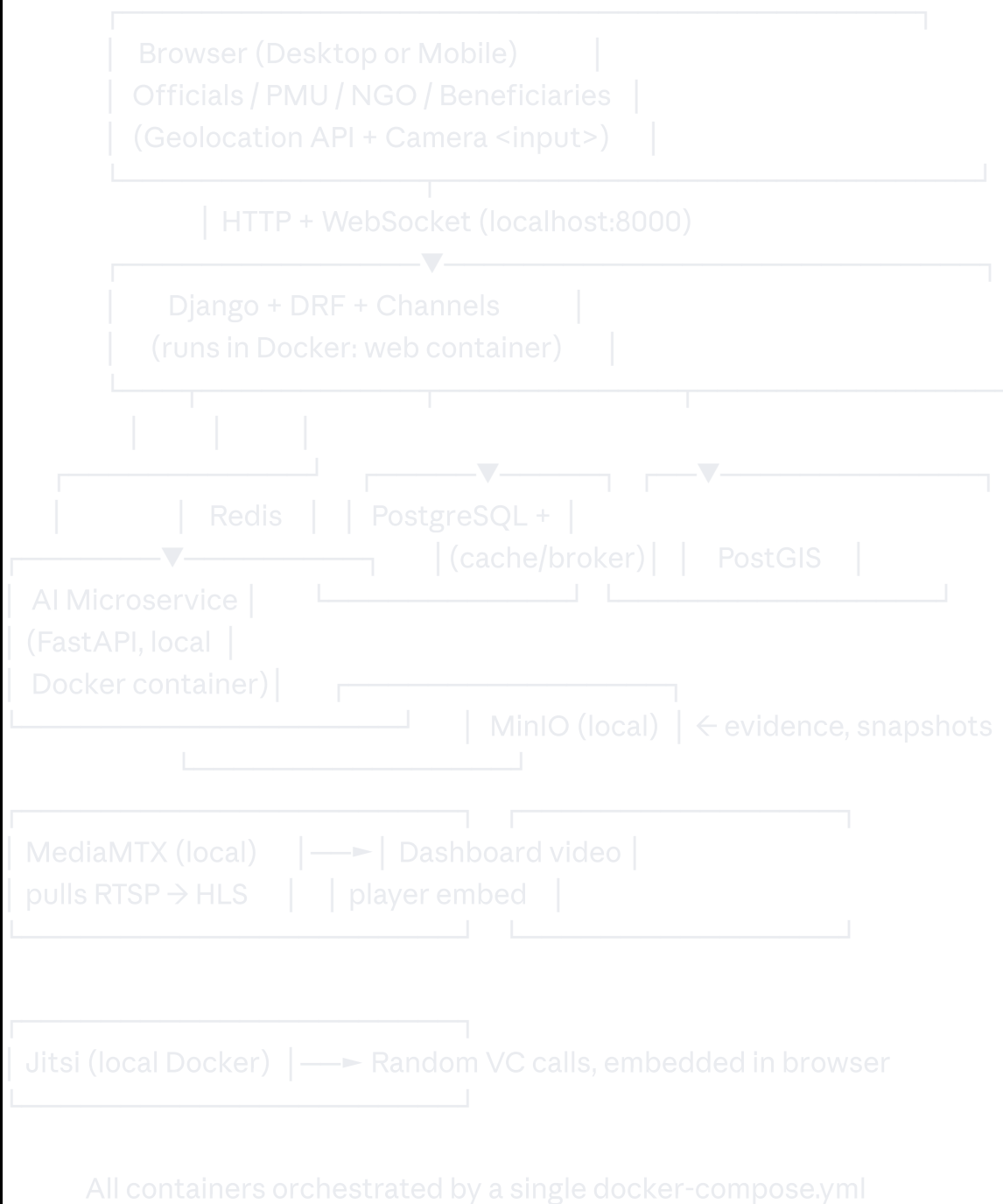
Layer	Choice	Why
Backend framework	Django + Django REST Framework (DRF)	Built-in admin panel, auth, ORM — fast to build and demo
Frontend (all roles)	Django Templates + HTMX + Alpine.js , OR a separate React (Vite) SPA	Since there's no native app, the web frontend must be responsive enough to be used comfortably from a phone browser by inspection officers in the field. HTMX keeps everything in one Django codebase — simplest for solo/local dev. React SPA if your team wants a richer dashboard/UX.
Real-time layer	Django Channels + Redis	Live dashboard updates, live notification banners, live CCTV/VC status
Task queue	Celery + Redis (broker)	Random assignment jobs, scheduled random VC triggers, snapshot capture, report generation
Database	PostgreSQL + PostGIS	Geo-tagging, geofence validation queries

Layer	Choice	Why
Video conferencing	Jitsi Meet (self-hosted via Docker)	Free, open-source, embeds directly into the browser — no app needed
CCTV ingestion	MediaMTX (RTSP → HLS) running locally in Docker	Institutes' CCTV RTSP URLs get converted to a stream your Django dashboard can <code><video></code> -embed
File/evidence storage	MinIO (S3-compatible, runs locally in Docker)	Evidence photos/videos, CCTV snapshots — swap for AWS S3 later with zero code change
Geo-tagging capture	Browser Geolocation API + HTML5 <code><input capture></code> for camera	No native app needed — modern mobile browsers support GPS + camera capture directly
AI/ML service	FastAPI microservice (Python, scikit-learn/OpenCV/YOLOv8)	Random-assignment fairness engine, anomaly detection, attendance/crowd analytics — runs as a separate local Docker container, called via REST from Django
Notifications	Django Channels (in-app) + Email (SMTP, e.g. local Mailhog for dev)	No FCM needed without a native app — in-app + email covers it for now; SMS/WhatsApp listed as an add-on feature below
Auth	Django's built-in auth + django-allauth (optional) , session-based for a pure web app (JWT only needed if you later expose APIs to a separate mobile app)	Simpler for a web-only local build
Local orchestration	Docker Compose	One command (<code>docker compose up</code>) brings up Postgres, Redis, Django, Celery worker, Celery beat, MediaMTX, MinIO, Jitsi, AI service — all on localhost
CI/CD (later)	GitHub Actions	Deferred until you deploy beyond local

Removed for now: Flutter/React Native mobile app, FCM push notifications, offline-first mobile sync — all mobile-specific concerns. Everything else that used to be "mobile app features" is delivered as a **mobile-responsive web page** instead, so no feature is actually lost — only the native app wrapper.

PART 3 — High-Level Architecture (Localhost)





PART 4 — Core Modules (from the original problem statement)

4.1 Auth & Role-Based Access Control

Roles: SuperAdmin (DoSJE HQ), StateAuthority, DistrictAuthority, PMU Team, Inspection Officer, NGO/Institute Admin, Project Incharge, Beneficiary.

4.2 Project/Institute/NGO Registry

Scheme, Project, Institute, NGO, Staff, Beneficiary — each geo-located, browsable on a map view in the dashboard.

4.3 Live CCTV Feed Integration

Institutes register RTSP URLs → MediaMTX republishes as HLS → embedded <video> player on the web

dashboard. Scheduled snapshot capture (Celery beat) stored in MinIO for AI scanning.

4.4 Random Video Conferencing

Celery Beat picks a random weighted subset of projects daily → generates a Jitsi room link → in-app notification + email to Incharge/Staff/Beneficiary → join via browser, no app install needed.

4.5 Real-Time Monitoring Dashboard

Django Channels pushes live status (CCTV up/down, ongoing VC, inspection status, alerts) to a map + card-based dashboard for officials.

4.6 Web-Based Inspection Module (replaces the mobile app)

Dynamic checklist forms (`InspectionTemplate`/`InspectionField`) — editable by DoSJE without code changes).

Photo/video evidence captured via `<input type="file" capture="environment">` (works on any phone browser camera).

GPS captured via browser Geolocation API, validated server-side against the registered project's coordinates (geofence check via PostGIS).

Digital signature/OTP confirmation from Project Incharge at closure.

Progressive Web App (PWA) wrapper (manifest + service worker) so officers can "Add to Home Screen" for an app-like feel — **without building or maintaining a separate native app.**

4.7 Random Inspection Assignment Engine (AI/automation)

Weighted random lottery (not pure uniform) based on: days since last inspection, prior anomaly/risk score, complaint count, officer's current workload/proximity. Every assignment logs its random seed + weights used — an auditable, un-riggable trail. Runs as a Celery periodic task.

4.8 Geo-Tagged Inspection Reports & Evidence

Every submission geo-validated against the project's registered location; auto-generated PDF report (WeasyPrint) combining form answers + evidence, downloadable by officials.

4.9 AI-Based Anomaly & Attendance Analytics

CCTV snapshot → person-count model (YOLOv8n) compared against expected roster → proxy-attendance flag. Statistical anomaly detection (Isolation Forest / z-score) on camera-uptime, VC no-show rate, inspection-score trend.

4.10 Notifications & Alerts

In-app (Channels) + email for now; SMS/WhatsApp listed below as an add-on once you're ready to integrate a paid gateway.

PART 5 — Additional Features (new, beyond the original PS — for extra judging points)

These strengthen the "transparency," "citizen-centric," and "innovation" criteria the PS explicitly calls out:

Public Transparency Portal — a read-only public web page (no login) showing aggregate, anonymized scheme performance per district/state (e.g. "% institutes inspected this quarter," "average compliance score") — directly serves the "Enhanced citizen-centric service delivery" outcome and is a strong differentiator most teams won't build.

Grievance/Complaint Management Module — beneficiaries can file a complaint against a project/institute directly from a public form; auto-routes to the relevant PMU team and factors into that institute's risk-weighting for the next random inspection.

Fund Utilization & Budget Tracking Dashboard — Institutes log expenditure against sanctioned scheme budget; flags mismatches between claimed activity and CCTV/inspection evidence (e.g. claimed 50 beneficiaries fed, but attendance analytics shows 20).

Escalation & SLA Workflow — if an anomaly flag or complaint isn't resolved within X days, it auto-escalates up the hierarchy (District → State → HQ) with a visible countdown on the dashboard.

Multi-language UI — Django's built-in i18n framework (`gettext`) for Hindi + regional language support, since NGO staff/beneficiaries across states will use this.

QR-Code Site Verification — each institute gets a printed QR code on-site; inspection officer scans it (via browser camera) as a secondary proof-of-presence alongside GPS — cheap, robust fallback when GPS is unreliable indoors.

Immutable Audit Log — use `django-simple-history` (or an append-only ledger table) on all inspection reports, assignments, and score changes, so nothing can be silently edited — pitch this as a lightweight "blockchain-style" integrity guarantee without the actual complexity of a blockchain.

Automated Weekly/Monthly PDF & Excel Report Generator — one-click export of district/state performance reports for officials (via WeasyPrint + openpyxl), reducing manual paperwork — directly serves "better inspection governance."

Chatbot/FAQ Assistant (optional stretch goal) — simple rule-based or LLM-backed assistant answering beneficiary FAQs about scheme eligibility/status, embedded on the public portal.

Document/Compliance Repository — central store for MoUs, NGO registration certificates, past audit reports, with expiry-date reminders (e.g. NGO registration renewal due) — a very "real government need" that judges will recognize.

PART 6 — Django Project Structure



```
dosje_platform/
├── config/          # settings, urls, asgi.py, celery.py
├── apps/
│   ├── accounts/    # custom user, roles, permissions
│   ├── registry/    # Scheme, Project, Institute, NGO, Staff, Beneficiary
│   ├── cctv/        # CameraDevice, StreamSession, Snapshot
│   ├── vc/          # VCSession, random-trigger scheduler
│   ├── inspections/ # InspectionTemplate, InspectionField, InspectionReport, Evidence
│   ├── assignment_engine/ # random assignment logic + AssignmentLog (audit)
│   ├── analytics/   # ComplianceScore, AnomalyFlag, dashboard aggregation
│   ├── grievances/  # Complaint, Escalation, SLA tracking [NEW]
│   ├── finance/     # Budget, Expenditure, mismatch flags [NEW]
│   ├── documents/   # NGO docs, MoUs, expiry reminders [NEW]
│   ├── transparency/ # public read-only views/APIs [NEW]
│   ├── notifications/ # in-app + email dispatch
│   └── core/        # shared utils, permissions, geofence validators
├── ai_service/      # standalone FastAPI microservice (separate Docker container)
│   ├── anomaly/
│   └── assignment_fairness/
├── static/ , templates/ # HTMX/Alpine frontend (if not using a separate React SPA)
├── docker-compose.yml  # postgres, redis, django, celery worker+beat, mediamtx, minio, jitsi,
ai_service
└── requirements.txt
```

PART 7 — Core Data Models

`User` (role, phone, preferred_language)

`Scheme` → `Project` / `Institute` / `NGO` (name, location: `PointField`, district, state, contact)

`CameraDevice` (institute FK, rtsp_url, status, last_seen)

`VCSession` (project FK, scheduled_time, participants, join_status, recording_url)

`InspectionTemplate` / `InspectionField` (dynamic checklist builder)

`InspectionAssignment` (officer FK, project FK, assigned_at, due_date, status, random_seed, weight_snapshot)

`InspectionReport` (assignment FK, submitted_at, geo_point, findings JSON, score, qr_verified bool)

`Evidence` (report FK, file_url, media_type, geo_point, captured_at)

`ComplianceScore` (project FK, score, last_updated, contributing_factors JSON)

`AnomalyFlag` (project FK, type, severity, detected_at, resolved_bool)

`Complaint` (beneficiary FK, project FK, description, status, escalation_level, sla_due_date) — **NEW**

`Budget` / `Expenditure` (project FK, sanctioned_amount, spent_amount, mismatch_flag) — **NEW**

`ComplianceDocument` (ngo FK, doc_type, file_url, expiry_date) — **NEW**

PART 8 — Development Roadmap (Local-first, phased)

Phase 0 — Local Environment Setup (Day 1)

`docker-compose.yml` with Postgres, Redis, Django skeleton.

Custom `User` model + role scaffolding.

Django admin working end-to-end (instant CRUD for every model — huge time-saver for local dev/demo).

Phase 1 — Registry + Auth (Days 2–3)

Project/Institute/NGO/Staff/Beneficiary registry with map view (Leaflet, using local dummy geo-data).

Role-based dashboards (even bare-bones) per user type.

Phase 2 — Inspection Module (Days 3–5)

Dynamic checklist builder, web-based evidence capture (camera + GPS via browser APIs), geofence validation, PDF report generation.

Phase 3 — Random Assignment Engine (Days 5–6)

Weighted lottery algorithm + Celery periodic task + audit log + dashboard view of "why this officer was picked."

Phase 4 — CCTV + VC (Days 6–8)

MediaMTX local container + a looped demo RTSP stream (e.g. `ffmpeg` re-streaming a sample video file) for reliable offline demoing.

Jitsi local container embedded via `iframe`; random-VC Celery beat job.

Phase 5 — AI Analytics (Days 8–10)

FastAPI microservice: rule-based anomaly detection first (fast to build), upgrade to YOLOv8n person-count / Isolation Forest if time allows.

Phase 6 — Additional Features (Days 10–13, pick based on remaining time)

Priority order for impact-per-effort: Grievance module → Public Transparency Portal → Audit log (django-simple-history) → PDF/Excel report export → QR verification → Fund tracking → Multi-language → Chatbot (last, optional).

Phase 7 — Polish & Demo Prep (Day 13+)

Seed realistic demo data, rehearse the "random assignment → inspection → evidence → dashboard alert" flow end-to-end, prepare the audit-log screen as your "innovation" talking point.

PART 9 — Team Role Split (adjusted — no mobile dev needed now)

Member	Focus
1–2	Django backend + DRF + Celery + DB/PostGIS design

Member	Focus
1	Django Channels (real-time) + frontend (HTMX/React) for dashboard & inspection forms
1	CCTV (MediaMTX) + VC (Jitsi) integration + Docker Compose orchestration
1	AI/ML: assignment fairness algorithm + anomaly detection microservice
1	Additional features (grievances, transparency portal, reports, docs) + demo data + PDF/Excel export

PART 10 — Security & Compliance Notes

All evidence/CCTV data encrypted at rest (MinIO server-side encryption) even in local dev, so the pattern carries to production unchanged.

Role-based data isolation — a district authority never sees another district's raw data unless escalated.

Immutable audit trail (Part 5, item 7) on every assignment, report edit, and score change.

Data retention policy stated explicitly for CCTV snapshots (mention in pitch — shows govt-readiness).

Since it's local-only for now: use `.env` files (never commit secrets), and design the settings so switching from local Postgres/MinIO to a cloud/NIC deployment later is a config change, not a rewrite.

PART 11 — Why This Plan Still Wins on Judging Criteria

Feasibility: everything runs with `docker compose up` on one laptop — no cloud accounts, no app-store builds, no native mobile toolchain to fight with during the hackathon.

No feature loss: dropping the native app doesn't drop any capability — camera/GPS/PWA-installable web app covers the same ground for a demo and even for real early-stage rollout.

Innovation: weighted+auditable random assignment, public transparency portal, and fund-utilization cross-checking are concrete differentiators beyond the base PS.

Scalability story: modular Django apps + a separate AI microservice + Docker Compose today, Kubernetes/cloud tomorrow — the architecture doesn't need to change, only where it's hosted.

Cost: every component (Django, Postgres, Redis, Jitsi, MediaMTX, MinIO) is open-source — zero licensing cost, a strong point for a government pitch.

Next step: I can scaffold the actual `docker-compose.yml` + Django project (apps, models, DRF serializers/viewsets) for Phase 0–1, or start with the random-assignment algorithm code — your call.